FinPlan — Documentação da API

Guia completo de funcionalidades, dependências, regras de negócio e validações · Versão do projeto: Next.js 16 + SQLite local

1. Visão geral

O **FinPlan** ("Seu futuro financeiro em boas mãos") é uma aplicação web de planejamento financeiro pessoal. A pessoa cadastra suas rendas e gastos, monta metas (caixinhas), planos de quitação de dívidas e uma reserva de emergência, e recebe análises e dicas calculadas por um motor de regras de negócio.

O que chamamos de **API** neste documento é o conjunto formado por:

- as **rotas HTTP** em `/api/...` (autenticação, conta, configuração e dados do usuário);
- o **motor de regras** (`lib/rules.ts`, `lib/reserve.ts`, `lib/calcConfig.ts`, `lib/ruleInsights.ts`) que avalia o orçamento, a reserva e os prazos;
- o **banco de dados local** (SQLite) que guarda tudo.

Princípios do projeto:

- **Local primeiro:** o banco roda na máquina de quem executa o app. Nada é enviado para a nuvem.
- **Cada pessoa, uma realidade:** todos os parâmetros das análises (50/30/20, reserva, prazos) são editáveis na Base de Cálculo.
- **Regras no servidor:** o que protege a integridade dos dados (categorias padrão, reserva única, validações) é verificado no servidor, e não só na tela.

2. Tecnologias e dependências

Requisitos: Node.js 22 ou superior (testado com 22.22) e npm.

Framework e interface

- `next` 16.4.3: App Router, Route Handlers (a API), Turbopack e o arquivo `proxy.ts` (que substitui o antigo *middleware*).
- `react` e `react-dom` 19.3.0: interface.
- `typescript` 5: tipagem em todo o código.
- `tailwindcss` 4 com `@tailwindcss/postcss`: estilos.
- `lucide-react`: ícones.
- `recharts`: gráficos do Dashboard.

Servidor e segurança

- `better-sqlite3` 12: banco SQLite embutido. É síncrono e é um módulo nativo (no Windows costuma baixar um binário pronto; se a instalação falhar, instale as ferramentas de compilação do Node).
- `bcryptjs` 3: criptografia (hash) das senhas.
- `jose` 6: criação e verificação do token de sessão (JWT).
- `nodemailer` 7: envio do e-mail de recuperação de senha.

Desenvolvimento: `eslint` 9 com `eslint-config-next`, `postcss` e os pacotes `@types/*`.

Variáveis de ambiente (arquivo `.env.local`, modelo em `.env.example`; todas opcionais):

- `AUTH_SECRET`: segredo que assina as sessões. Se vazio, é gerado em `data/auth-secret.key`.
- `APP_URL`: endereço usado no link do e-mail de recuperação (padrão `http://localhost:3000`).
- `COOKIE_SECURE`: use `true` somente se o app for servido por HTTPS.
- `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM`: e-mail de recuperação. Sem SMTP, o link aparece no terminal do servidor.
- `DB_PATH`: caminho alternativo do arquivo do banco.

3. Como o sistema foi construído

- **Aplicação única:** o Next.js serve as páginas e a API no mesmo processo. Não há servidor separado.
- **Páginas:** em `app/`. A pasta `app/(app)/` agrupa as telas com a barra lateral (Planejamento, Previsões, Configurações).
- **API:** Route Handlers em `app/api/`.
- **Proteção de rotas:** `proxy.ts` roda antes de cada requisição.
- **Lógica pura compartilhada:** os arquivos de regras em `lib/` não dependem de React nem do banco, então valem tanto no servidor quanto no navegador.
- **Acesso a dados no navegador:** o hook `useUserData(kind)` lê, cria, edita e apaga registros pela API, com atualização imediata na tela e desfazer em caso de erro.
- **Desenvolvimento:** o projeto foi construído de forma iterativa com o auxílio do assistente de IA Claude (Anthropic), e verificado com o compilador TypeScript, o ESLint e scripts de teste de lógica e de requisições HTTP.

4. Instalação e execução

1. `npm install`
2. (opcional) copie `.env.example` para `.env.local` e preencha.
3. `npm run dev` e abra `http://localhost:3000`.
4. Para produção local: `npm run build` e depois `npm start`.

Observações: o build baixa as fontes Geist do Google, então precisa de internet nessa etapa. O banco e o segredo são criados sozinhos na pasta `data/` (já ignorada pelo Git).

Backup: pare o app e copie os arquivos `data/finplan.db*` (o SQLite usa também arquivos `-wal` e `-shm`).

5. Estrutura de pastas

- `app/`: páginas (`/`, `/login`, `/redefinir-senha`, `/dashboard`, `/planejamento`, `/planejamento/renda`, `/previsoes`, `/configuracoes`) e `api/`.
- `components/`: telas e peças visuais (auth, dashboard, planejamento, previsoes, configuracoes).
- `lib/`: banco, sessão, regras de negócio, validações e hooks.
- `proxy.ts`: controle de acesso às páginas e às APIs.
- `data/`: banco `finplan.db` e `auth-secret.key` (criados automaticamente).

6. Banco de dados

SQLite em `data/finplan.db`, com modo WAL e chaves estrangeiras ligadas. Ao apagar um usuário, seus dados somem em cascata.

- **users:** id, nome, sobrenome, sexo (`masculino`, `feminino` ou `nao_informado`), usuário e e-mail (únicos, sem diferenciar maiúsculas), hash da senha e data de criação.
- **password_resets:** token (guardado só como hash SHA-256), validade e marca de uso.
- **user_data:** tudo que o usuário insere. Cada linha tem `user_id`, o tipo (`kind`), o conteúdo em JSON, e datas de criação e edição.
- **calc_config:** a Base de Cálculo de cada usuário, em versões (ver seção 10).
- **user_flags:** marcas internas, como "categorias padrão já criadas".

Tipos (`kind`) usados em `user_data`:

- `categorias`, `rendas`, `gastos-fixos`, `gastos-variaveis`, `caixinhas`, `dividas` e `reserva`.

Todas as consultas filtram por `user_id`. Um usuário nunca lê nem altera dados de outro.

7. Autenticação e segurança

- **Senhas:** bcrypt com custo 12, mínimo de 8 caracteres. A senha nunca é devolvida pela API.
- **Sessão:** token JWT (HS256) válido por 7 dias, guardado no cookie `finplan_session` com `httpOnly` e `SameSite=Lax`.
- **Controle de acesso (`proxy.ts`):** sem login, só `/`, `/login` e `/redefinir-senha` abrem. Qualquer outra página redireciona para `/login?next=...`. APIs fora de `/api/auth/` respondem 401. Quem já está logado e abre `/login` volta para o início.
- **Defesa em profundidade:** cada rota de dados confere a sessão de novo e filtra por usuário.
- **Tentativas de login:** bloqueio por 10 minutos após 5 erros (por usuário e IP). A alteração de dados da conta tem o mesmo limite. O contador fica na memória e zera ao reiniciar.
- **Usuário inexistente:** a API gasta o mesmo tempo de uma senha errada, para não revelar quais usuários existem.
- **Recuperação de senha:** token aleatório de 32 bytes, guardado apenas como hash, válido por 1 hora e de uso único. A resposta é sempre a mesma, exista ou não o e-mail.
- **Redirecionamento após o login:** só aceita caminhos internos.
- **Links externos:** abrem em nova aba com `noopener noreferrer`.

8. Rotas da API

Todas respondem JSON. Erros comuns: 400 (dados inválidos), 401 (não autenticado), 403 (proibido), 404, 409 (conflito), 413 (grande demais) e 429 (muitas tentativas).

Autenticação

- `POST /api/auth/register`: cria a conta, já cria as categorias padrão e a Base de Cálculo, e inicia a sessão.
- `POST /api/auth/login`: entra com usuário **ou** e-mail e senha.
- `POST /api/auth/logout`: encerra a sessão.
- `GET /api/auth/me`: devolve o usuário logado.
- `POST /api/auth/forgot-password`: envia o link de nova senha.
- `POST /api/auth/reset-password`: define a nova senha com o token.

Conta

- `PATCH /api/account`: altera nome, sexo, usuário, e-mail e (opcionalmente) a senha. Exige a **senha atual**.

Base de Cálculo

- `GET /api/config`: devolve as versões, o valor que vale hoje, o último valor e a alteração agendada (se houver).
- `PUT /api/config`: salva os valores com `scope all` ou `next-month`.
- `DELETE /api/config`: cancela a alteração agendada.

Dados do usuário

- `GET /api/data/:kind`: lista os registros do tipo.
- `POST /api/data/:kind`: cria um registro.
- `PUT /api/data/:kind/:id`: substitui o registro.
- `DELETE /api/data/:kind/:id`: apaga o registro.

9. Funcionalidades por tela

Início: mensagem de boas-vindas com o nome completo e a forma correta ("Bem-vindo", "Bem-vinda" ou "Bem-vindo(a)", conforme o sexo informado) e atalhos para as áreas. Sem login, qualquer clique leva ao login.

Login e cadastro: entrada por usuário ou e-mail. O cadastro e a recuperação de senha abrem em janelas sobre a própria tela de login.

Dashboard: indicadores (receita média mensal, gasto médio mensal, percentual de saúde financeira e saldo do período), gráficos de evolução e distribuição por categoria, lista de despesas específicas e filtro de período (este mês, mês passado, últimos 3 ou 6 meses, este ano ou período específico). Tudo é calculado a partir do que está no banco.

Planejamento

- *Categorias:* cada uma tem nome, ícone, cor e **grupo** (Necessidade, Desejo ou Investimento). As padrão vêm com a conta e não podem ser apagadas.
- *Rendas:* recorrentes (com início e fim opcionais) ou pontuais, com responsável e observações.
- *Gastos fixos:* valor mensal por categoria, com duração de 1 a 12 meses ou indeterminada, a partir do mês escolhido.
- *Gastos variáveis:* gastos com data, nome, valor e categoria.

Previsões

- *Reserva de Emergência:* ver seção 10.
- *Caixinha (Objetivos):* meta, valor já guardado e data. A API calcula quanto guardar por mês. Cada caixinha mostra a etiqueta de prazo (curto, médio ou longo).
- *Finalize Dívidas:* plano de quitação com juros e data final, com parcela mensal e juros estimados.
- *Dicas da API & Análise Inteligente:* os cartões da seção 10.

Configurações

- *Meus dados:* edita nome, sexo, usuário, e-mail e senha. Só salva com a senha atual.
- *Legislação:* leis de referência em resumo, com link para o texto oficial. Hoje traz a Lei do Superendividamento (Lei nº 14.181/2021). Novas leis entram em `lib/legislation.ts`.
- *Base de Cálculo:* tabela editável com os parâmetros das análises e o botão **Base de Dados** (seção 10).

10. Regras de negócio

10.1 Grupos de categorias. A API usa o grupo para avaliar o orçamento.

- Necessidades: Alimentação, Transporte, Moradia, Saúde, Educação, Serviços e Dívidas.
- Desejos: Lazer, Vestuário e Outros.
- Investimento: Investimentos.

Essas 11 categorias são criadas com a conta, não podem ser apagadas nem trocar de grupo (podem ser renomeadas, com novo ícone ou cor). Categorias criadas pelo usuário exigem a escolha do grupo. Um gasto sem categoria conhecida conta como Desejo.

10.2 Orçamento 50/30/20. Para o mês atual, a API soma a renda (considerada líquida) e os gastos de cada grupo, e compara com a Base de Cálculo:

- Necessidades e Desejos têm **teto** (padrão 50% e 30% da renda).
- Poupança e Investimento têm **piso** (padrão 20%).
- Entram como Investimento os gastos da categoria Investimentos **e** os aportes do mês da reserva e das caixinhas em andamento.
- A comparação usa percentuais inteiros (os mesmos que aparecem na tela). Mês sem renda não é avaliado.
- Resultado: dentro da regra (verde), 1 alerta (amarelo) ou 2 ou mais alertas (vermelho), com o valor em reais acima do teto ou abaixo do piso.

10.3 Reserva de Emergência. Funciona como uma caixinha, com regras próprias:

- **Ativação:** desativada, não entra em nenhum cálculo nem nas dicas.
- **Quanto já possui:** nada, 6 meses, 2 anos ou 5 anos.
- **Início:** o mês em que os aportes começam ou continuam (do mês atual em diante).
- **Medida:** cobertura em meses das **despesas fixas mensais**. Os marcos são 6 meses, 24 meses e 60 meses.
- **Aportes:** cada aporte mensal entra no valor guardado quando o mês termina. Meses em pausa não contam.
- **Pausar e retomar:** a pausa vale a partir do mês seguinte. Retomar volta a contar a partir do mês atual. Se a pausa ainda não começou, retomar apenas a cancela.
- **Marco concluído:** vale o maior entre o que a pessoa declarou e o que o cálculo mostra. Ao atingir um marco novo, aparece um aviso no topo das Dicas, que fica até a pessoa clicar em "Entendi". Marcos que ela já declarou possuir não geram aviso.
- **Faixa recomendada:** a API compara a cobertura com o intervalo da Base de Cálculo (padrão de 6 a 12 meses de despesas fixas).

10.4 Horizontes de tempo. Curto prazo até 12 meses, médio de 13 a 60 e longo acima de 60 (padrão). Caixinhas e dívidas recebem a etiqueta. Uma dívida com prazo acima do curto prazo gera uma dica.

10.5 Base de Cálculo versionada. Os valores (todos números inteiros) ficam em versões, cada uma com o mês em que passa a valer:

- **Todas as informações:** os novos valores valem para todo o histórico.
- **A partir do mês seguinte:** até o fim do mês atual valem os valores antigos, e o histórico não muda.
- Só existe uma alteração agendada por vez. Uma nova substitui a anterior, e é possível cancelar o agendamento.

- O botão **Base de Dados** obriga a escolher uma das duas formas e mostra o resumo do que mudou.
- Todo usuário nasce com os valores padrão (50/30/20, reserva de 6 a 12 meses, horizontes de 12 e 60 meses).

10.6 Dicas da API. Ordem de exibição: aviso de reserva concluída, orçamento, saúde financeira, tendência, maior despesa, economia identificada, situação da reserva e prazo de dívida. Mais a recomendação estratégica.

- *Saúde financeira:* gastos abaixo de 50% da renda = saudável; de 50% a 79% = moderado; 80% ou mais = atenção.
- *Tendência:* últimos 3 meses contra os 3 anteriores.
- *Economia identificada:* categoria que mais caiu contra o mês anterior.
- *Recomendação:* cortar 10% da maior categoria e levar para a caixinha mostra o acúmulo em 12 meses.

11. Fórmulas

- **Meses até uma data:** dias até a data divididos por 30,4375, arredondado para cima, com mínimo de 1.
- **Caixinha:** parcela = (meta − já guardado) ÷ meses.
- **Dívida sem juros:** parcela = saldo ÷ meses.
- **Dívida com juros (tabela Price):** parcela = saldo × r ÷ (1 − (1 + r)^−n), com r = taxa mensal e n = meses. Juros estimados = parcela × n − saldo.
- **Transações do Dashboard:** rendas e gastos fixos recorrentes viram um lançamento por mês, até o mês atual (limite de 120 meses). Rendas pontuais futuras não entram.
- **Renda base (Previsões):** média da renda dos últimos 6 meses.
- **Índice de saúde do Dashboard:** 100 até os gastos chegarem a 50% da renda, caindo em linha reta até 0 quando igualam a renda. É uma fórmula provisória, ajustável em `lib/dashboard.ts`.

12. Validações

Cadastro (servidor)

- Todos os campos obrigatórios; sexo entre as três opções.
- E-mail em formato válido (gravado em minúsculas).
- Usuário com 3 a 30 caracteres: letras, números, ponto, hífen ou sublinhado.
- Senha com no mínimo 8 caracteres e igual à confirmação.
- Usuário e e-mail únicos (erro 409).

Conta (`PATCH /api/account`)

- Senha atual obrigatória e correta (erro 403 se errada; 429 após 5 erros).
- Mesmas regras do cadastro. A nova senha é opcional.
- Usuário e e-mail não podem pertencer a outra conta.

Categorias

- Nome de 1 a 24 caracteres, ícone de até 30 caracteres, cor no formato `#RRGGBB` e grupo válido.
- Categoria padrão: grupo e marca fixos; apagar retorna 403.

- O cliente não consegue criar categoria marcada como padrão.

Reserva

- Só pode existir uma por usuário (409 ao criar outra).
- Ativa (sim/não), quanto possui, marco dispensado, valor guardado e aporte maiores ou iguais a zero, mês de início no formato `aaaa-mm`, pausas bem formadas.

Base de Cálculo

- Sete valores, todos **inteiros** e maiores ou iguais a zero: três percentuais (até 100), mínimo e máximo de meses da reserva, e os limites do curto e do médio prazo (até 1200).
- Decimais, textos e negativos são recusados (na tela e no servidor).
- Escolher como aplicar (todas as informações ou mês seguinte) é obrigatório.
- Por enquanto não há outras restrições: a soma dos percentuais pode ser diferente de 100 (a tela só avisa). Os mínimos dos horizontes são calculados a partir do limite anterior, sem lacunas.

Valores e formulários

- Dinheiro aceita o formato brasileiro (1.234,56) e precisa ser maior que zero; campos opcionais vazios valem zero.
- Juros: de 0 a 100% ao mês.
- Caixinhas e dívidas exigem data futura.
- Gastos fixos: duração de 1 a 12 meses ou indeterminada.

Dados genéricos (`/api/data`)

- Tipo com 1 a 40 caracteres (letras minúsculas, números, hífen ou sublinhado); id numérico.
- Corpo precisa ser um objeto JSON de até 100.000 caracteres (413 se maior).
- Editar ou apagar só funciona em registros do próprio usuário.

13. Privacidade e limitações

Privacidade

- Os dados ficam só no arquivo local `data/finplan.db`. A única comunicação externa é o e-mail de recuperação, enviado pelo SMTP que você configurar. As fontes são baixadas na compilação.
- Os dados não são criptografados em repouso (só as senhas têm hash). Quem tiver acesso ao computador ou ao arquivo do banco consegue ler as informações. Proteja a máquina e os backups.

Limitações conhecidas

- **Não é consultoria.** As dicas e a aba Legislação são informativas e não substituem um profissional financeiro ou jurídico. A Lei do Superendividamento é apresentada como referência: o FinPlan não calcula nem declara superendividamento.
- **Premissas:** a renda cadastrada é tratada como líquida; os aportes da reserva e das caixinhas contam automaticamente como Investimento (evite lançar o mesmo aporte também como gasto da categoria Investimentos); a classificação das categorias padrão é uma decisão do projeto (ex.: Educação e Dívidas em Necessidades).
- **Sessões:** o token é sem estado. Trocar a senha não encerra sessões já abertas em outros navegadores, e não há verificação em duas etapas.
- **Limite de tentativas** em memória: zera ao reiniciar e não serve para vários servidores.

- **Um único servidor:** o SQLite local não foi pensado para vários servidores ao mesmo tempo.
- **Pendências:** o botão "Aplicar no Plano" da recomendação ainda não executa ação, e os cálculos de caixinha e dívida ainda rodam no navegador (a ideia é migrá-los para a Base de Cálculo).
- **Testes:** ainda não há suíte de testes automatizados versionada no repositório; as verificações feitas foram scripts pontuais.

14. Próximos passos sugeridos

- Versionar uma suíte de testes (regras, validações e rotas).
- Encerrar sessões ao trocar a senha e oferecer verificação em duas etapas.
- Mover os cálculos de caixinha e dívida para o servidor, usando a Base de Cálculo.
- Ligar o botão "Aplicar no Plano" a uma ação real.
- Incluir novas leis na aba Legislação e criptografar o banco em repouso, se o uso exigir.

15. Glossário

- **Renda líquida:** o que a pessoa realmente recebe, depois de descontos.
- **Regra 50/30/20:** divisão da renda em 50% necessidades, 30% desejos e 20% poupança e investimento. Aqui, os percentuais são editáveis.
- **Despesas fixas mensais:** gastos que se repetem todo mês (aluguel, escola, internet). São a base da reserva.
- **Reserva de emergência:** dinheiro separado para imprevistos, medido em meses de despesas fixas.
- **Mínimo existencial:** renda necessária para as despesas básicas de uma vida digna (conceito da Lei nº 14.181/2021).
- **Horizonte temporal:** classificação do prazo de uma meta em curto, médio ou longo prazo.

16. Operação em uso diário

Rodar no seu computador. Depois de `npm install`, use `npm run build` (só na primeira vez e a cada atualização) e `npm start`. No Windows, o `iniciar-finplan.bat` faz isso e pode ser colocado em `shell:startup` para abrir com o Windows. O `npm start` aceita conexões **só do próprio computador** (`127.0.0.1`). O `npm run start:rede` libera a rede local; use somente com HTTPS e `COOKIE_SECURE=true`, porque sem isso as senhas trafegam sem criptografia.

Onde ficam os dados. Por padrão em `data/` dentro do projeto. Em uso diário, o recomendado é definir `DB_PATH` e `AUTH_SECRET` no `.env.local`, guardando o banco fora da pasta do código (ex.: `C:\FinPlanDados\finplan.db`). Assim, trocar ou reinstalar o código nunca toca nas contas.

Backup. `npm run backup:db` cria uma cópia íntegra do banco (um único arquivo, mesmo com o app rodando) em `backups/`, mantendo as 30 mais recentes. Use `--dest` para outra pasta (ex.: um HD externo ou uma pasta sincronizada) e `--keep` para a quantidade. Para restaurar: pare o app, copie o backup por cima do arquivo do banco e apague `finplan.db-wal` e `finplan.db-shm`, se existirem.

Atualizar. Faça um backup, atualize o código, rode `npm install`, `npm run build` e `npm start`. Os dados não são afetados.

Publicar no GitHub sem contas. `npm run copia:limpa -- "pasta-destino"` gera uma cópia sem `node_modules`, `.next`, `data/`, `backups/`, `.env*` (exceto `.env.example`), arquivos `.db` e chaves, e confere o resultado antes de terminar. O `.gitignore` do projeto também bloqueia esses itens. Antes do primeiro envio, rode `git status` e confirme que nenhum desses arquivos aparece.

Três cópias, três finalidades.

- *Desenvolvimento:* sua pasta de trabalho completa (com `data/`, se quiser guardar as contas de teste). Fica só com você.
- *GitHub:* a cópia limpa, só código.
- *Em uso:* uma pasta separada, instalada a partir do GitHub ou da cópia limpa, com banco próprio e vazio.